PHASE

3

Segmentation, the IDT and CPU Exceptions

Install the kernel's own descriptor tables and turn every CPU fault into a readable crash report instead of a silent reboot

Depends on Phase 2: C kernel with `kprintf`, serial logging and `panic`

Delivers A kernel-owned Global Descriptor Table (flat model with kernel and user segments), an Interrupt Descriptor Table with 32 exception stubs in NASM, a common C handler that receives the full register state, and a crash screen that names the exception and dumps the registers.

1 Scope and motivation

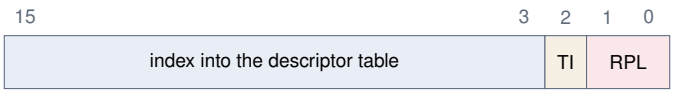
Without an IDT, a kernel that divides by zero or jumps to a bad address makes the CPU look for an exception handler, find none, fault again, and reset the machine. QEMU simply reboots. With this phase in place the same bug prints "Divide error (#DE) at EIP=0x00101234" with every register, which leads straight to the faulty line. No other change in the whole project saves as much debugging time.

The phase has two parts that belong together. The **GDT** defines the segments the CPU uses and, more importantly for Kagan, the privilege levels: ring 0 for the kernel and ring 3 for users. The Multiboot contract says GRUB's GDT may be invalid, so the kernel installs its own before touching a segment register. The **IDT** tells the CPU where to jump for each of the 256 interrupt vectors. Vectors 0–31 are CPU exceptions, handled in this phase. Vectors 32–47 will be hardware interrupts in Phase 4, and vector `0x80` becomes the system call gate in Phase 10. This is the *trap table* that OSTEP chapter 6 describes: the kernel installs it at boot, and from then on the hardware knows where to go when something needs the kernel's attention.

2 Background

2.1 Segmentation, and why Kagan neutralises it

In protected mode every memory access goes through a segment. A **segment selector** in CS, DS, SS and the other segment registers picks a **descriptor** from the GDT (Figure 1), and the descriptor gives the segment's base address, its limit, and its access rights. The CPU adds the base to every address and checks it against the limit. OSTEP chapter 16 explains the general idea, and OSDI §4.6.2 describes the Intel version.



TI = 0 selects the GDT. RPL = requested privilege level. Selector `0x08` = index 1, ring 0 (kernel code); `0x10` = index 2 (kernel data); `0x1B` = index 3 with RPL 3 (user code).

Figure 1: A 16-bit segment selector.

Modern operating systems use paging for memory protection, so they neutralise segmentation with the **flat model**: every descriptor has base 0 and a limit of 4 GiB, which makes logical addresses equal to linear addresses. The GDT still matters for three reasons. The CPU refuses to run in protected mode without one. The **privilege level** of running code is the DPL of its code segment, so ring 0 and

ring 3 need separate descriptors. And the **TSS** descriptor that Phase 10 needs for user mode lives in the GDT.

Each descriptor packs its fields into 8 bytes in a famously awkward layout (Figure 2), a leftover from the 286. The two bytes that matter are the **access byte** (present bit, DPL, code or data, readable or writable) and the **flags** nibble (4 KiB granularity and 32-bit operation). Table 1 lists the values Kagan uses.

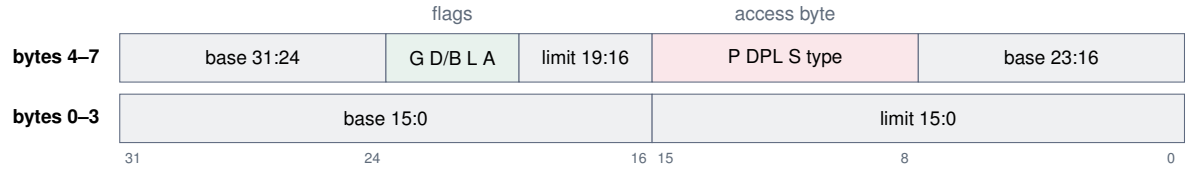


Figure 2: The 8-byte segment descriptor. Base and limit are scattered across both halves.

Index	Selector	Purpose	Access byte	Flags	Limit
0	`0x00`	null descriptor (required, never used)	`0x00`	`0x0`	0
1	`0x08`	kernel code: present, ring 0, executable, readable	`0x9A`	`0xC`	`0xFFFFF`
2	`0x10`	kernel data: present, ring 0, writable	`0x92`	`0xC`	`0xFFFFF`
3	`0x18`	user code: as kernel code but DPL 3 (Phase 10)	`0xFA`	`0xC`	`0xFFFFF`
4	`0x20`	user data: as kernel data but DPL 3 (Phase 10)	`0xF2`	`0xC`	`0xFFFFF`
5	`0x28`	task state segment (filled in Phase 10)	`0x89`	`0x0`	`sizeof(tss)-1`

Table 1: The Kagan GDT. Flags `0xC` = 4 KiB granularity + 32-bit, so limit `0xFFFFF` covers 4 GiB.

2.2 Interrupts, exceptions and traps

Three kinds of event make the CPU stop and run a handler. **Hardware interrupts** come from devices (Phase 4). **Exceptions** come from the CPU itself when an instruction can't complete. **Software interrupts** come from the `int n` instruction, which is how user programs will call the kernel. Exceptions are further divided by what happens on return (Table 2). After a **fault**, the saved EIP points at the faulting instruction, so returning retries it; that is how page faults will work in Phase 6. After a **trap**, EIP points past the instruction, so execution continues; `int3` works this way. An **abort** means the machine state is unreliable, and the only option is to report it and stop.

2.3 What happens on an interrupt

When vector n fires, the CPU reads IDT entry n. Each entry (a **gate**) holds the handler's address, the code segment selector to run it with (`0x08`), and a type byte. Kagan uses `0x8E`: present, DPL 0, 32-bit *interrupt gate*, which clears the interrupt flag on entry. A *trap gate* (`0x8F`) leaves interrupts enabled. The CPU then pushes EFLAGS, CS and EIP on the stack and, for some exceptions, an error code, and jumps to the handler. The handler returns with `iret`, which pops EIP, CS and EFLAGS back. (When the interrupt arrives in ring 3, the CPU also switches stacks and pushes the user SS and ESP; that is Phase 10's topic.)

C cannot be the handler directly: a C function does not know it must preserve every register and return with `iret`. So each vector gets a tiny assembly stub that makes the stack look the same for every interrupt, saves all registers, and calls one C function with a pointer to the saved state (Figure 3). Getting this layout exactly right, and matching it with a C struct, is the heart of the phase.

Vector	Mnemonic	Name	Class	Error code
0	#DE	Divide error	fault	no
1	#DB	Debug	fault or trap	no
2	–	Non-maskable interrupt	interrupt	no
3	#BP	Breakpoint (`int3`)	trap	no
4	#OF	Overflow (`into`)	trap	no
5	#BR	Bound range exceeded	fault	no
6	#UD	Invalid opcode	fault	no
7	#NM	Device (FPU) not available	fault	no
8	#DF	Double fault	abort	yes (always 0)
10	#TS	Invalid TSS	fault	yes
11	#NP	Segment not present	fault	yes
12	#SS	Stack-segment fault	fault	yes
13	#GP	General protection	fault	yes
14	#PF	Page fault (address in CR2)	fault	yes
16	#MF	x87 floating-point error	fault	no
17	#AC	Alignment check	fault	yes
18	#MC	Machine check	abort	no
19	#XM	SIMD floating-point	fault	no
21	#CP	Control protection	fault	yes

Table 2: The CPU exceptions Kagan handles. Vectors 9, 15, 20 and 22–31 are reserved or rare and get a generic handler.

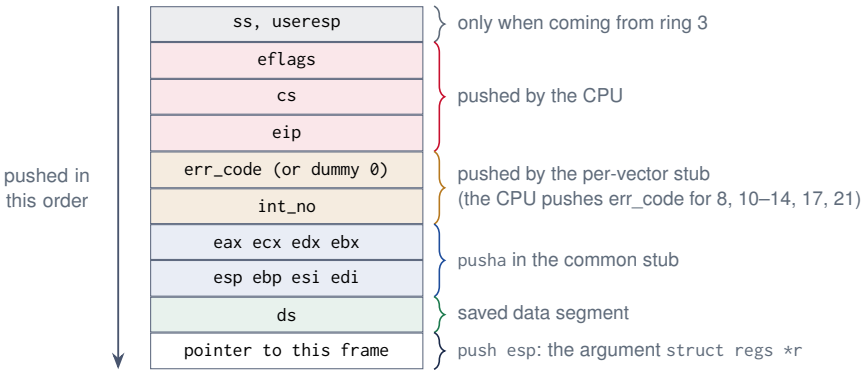


Figure 3: The stack when `isr_handler` is called. `struct regs` lists these fields from the bottom (`ds`) up.

3 References

Weight	Source	Section	Pages	Relevance
PRIMARY	LOB	Ch. 5 *Segmentation*: accessing memory, the GDT, loading the GDT	33–37	Tasks 3.1–3.2.
PRIMARY	LOB	Ch. 6 *Interrupts and Input*, up to and including *Loading the IDT*	39–43	Tasks 3.3–3.5; PIC and keyboard wait for Phase 4.
PRIMARY	OSTEP	Ch. 6 *Mechanism: Limited Direct Execution*	49–62	Why a trap table exists and what the hardware does on a trap.
PRIMARY	OSTEP	Ch. 16 *Segmentation*	149–160	The general idea behind the GDT descriptors.
SECONDARY	OSDI	§4.6.2 *Segmentation with Paging: The Intel Pentium*	415–420	Selectors, descriptors, GDT and LDT on real x86 hardware.
SECONDARY	MOS	§3.7.3 *Segmentation with Paging: The Intel x86*	247–252	A second explanation, with the history of why x86 kept segments.
SECONDARY	MOS	§5.1.5 *Interrupts Revisited*	347–351	Precise and imprecise interrupts; what “saved state” means.
SECONDARY	OSDI	§2.6.8 *Interrupt Handling in MINIX 3*	167–178	A complete, working implementation of this phase’s design.
OPTIONAL	OSC	§1.2 *Computer-System Organization* (interrupts); §17.3 *Protection Rings*	7–15, 669–671	Interrupt mechanics and rings in general terms.
SECONDARY	LKP	Ch. 6, *Understanding process and interrupt contexts*	268–270	The two contexts handlers run in, and what each may do.

The authoritative source is the Intel SDM Volume 3A: the parts of chapter 3 (protected-mode memory management) about segment selectors and descriptors, and chapter 6 (interrupt and exception handling), especially the exception classification, the error code format, and the per-exception reference at its end, which is the reference for the handler names.

Prior art: MINIX 3

MINIX builds its GDT and IDT in `kernel/protect.c (08300)`; look for `prot_init`, which fills the tables from a list of gate descriptions much like `idt_init` in Task 3.3. The entry stubs are macros in `kernel/mpx386.s (06300)`, and exceptions end up in `kernel/exception.c (08000)`. That file shows the decision Kagan makes in Phase 10: an exception in a user process becomes a signal to that process (for example, divide error sends `SIGFPE`), while an exception inside the kernel is a panic.

Linux, for comparison. LKD Linux keeps the flat model too: its GDT holds kernel and user code and data segments, the TSS and per-CPU entries, and every exception goes through the same kind of stub table into C handlers.

4 Implementation

Task 3.1 Describe the GDT in C

Write `kernel/arch/i386/gdt.c`. Pack the structs (they are hardware formats), fill the five entries of Table 1, and leave room for the TSS.

kernel/arch/i386/gdt.c

```c
struct gdt_entry {
    uint16_t limit_low;
    uint16_t base_low;
    uint8_t  base_mid;
    uint8_t  access;
    uint8_t  gran;          /* high nibble: flags (G, D/B); low nibble: limit 19:16 */
    uint8_t  base_high;
} __attribute__((packed));

struct gdt_ptr {
    uint16_t limit;         /* size of the table in bytes, minus one */
    uint32_t base;          /* linear address of the table           */
} __attribute__((packed));

static struct gdt_entry gdt[6];
static struct gdt_ptr   gp;
extern void gdt_flush(struct gdt_ptr *p);   /* in gdt_flush.s */

static void gdt_set(int i, uint32_t base, uint32_t limit, uint8_t access, uint8_t flags)
{
    gdt[i].base_low  = base & 0xFFFF;
    gdt[i].base_mid  = (base >> 16) & 0xFF;
    gdt[i].base_high = (base >> 24) & 0xFF;
    gdt[i].limit_low = limit & 0xFFFF;
    gdt[i].gran      = ((limit >> 16) & 0x0F) | (flags << 4);
    gdt[i].access    = access;
}

void gdt_init(void)
{
    gdt_set(0, 0, 0,       0x00, 0x0);  /* null                        */
    gdt_set(1, 0, 0xFFFFF, 0x9A, 0xC);  /* 0x08 kernel code            */
    gdt_set(2, 0, 0xFFFFF, 0x92, 0xC);  /* 0x10 kernel data            */
    gdt_set(3, 0, 0xFFFFF, 0xFA, 0xC);  /* 0x18 user code  (Phase 10)  */
    gdt_set(4, 0, 0xFFFFF, 0xF2, 0xC);  /* 0x20 user data  (Phase 10)  */
    /* entry 5 (0x28) becomes the TSS in Phase 10 */
    gp.limit = sizeof(gdt) - 1;
    gp.base  = (uint32_t)&gdt;
    gdt_flush(&gp);
}
```

Task 3.2 Load the GDT and reload the segment registers

`lgdt` only tells the CPU where the table is. The segment registers still hold the old descriptors in a hidden cache until they are reloaded. Data segments can be loaded with `mov`; CS can only be changed by a far jump (or a far call or `iret`).

kernel/arch/i386/gdt_flush.s

```asm
global gdt_flush
gdt_flush:
    mov  eax, [esp + 4]     ; argument: pointer to struct gdt_ptr
    lgdt [eax]              ; load the GDT register
    mov  ax, 0x10           ; 0x10 = kernel data selector
```

```
    mov  ds, ax
    mov  es, ax
    mov  fs, ax
    mov  gs, ax
    mov  ss, ax
    jmp  0x08:.reload_cs    ; far jump: CS = 0x08 (kernel code)
.reload_cs:
    ret
```

Verify: The kernel keeps running after `gdt_init()`. In the QEMU monitor, `info registers` shows a `GDT=` line whose base is the address of the `gdt` array (check with `i686-elf-nm`) and whose limit is `0000002f`; CS is `0008` and DS is `0010`.

Task 3.3 Build and load the IDT

Write `kernel/arch/i386/idt.c` with a 256-entry table of gates, a setter, and `idt_init`, which installs the 32 exception stubs from the next task and loads the table with `lidt`. Unfilled entries stay zero, which means "not present"; if such a vector fires the CPU raises a #GP, which the handler reports.

kernel/arch/i386/idt.c

```c
struct idt_entry {
    uint16_t offset_low;    /* handler address bits 15:0            */
    uint16_t selector;      /* code segment to run it in: 0x08      */
    uint8_t  zero;
    uint8_t  type_attr;     /* 0x8E = present, ring 0, 32-bit interrupt gate */
    uint16_t offset_high;   /* handler address bits 31:16           */
} __attribute__((packed));

struct idt_ptr { uint16_t limit; uint32_t base; } __attribute__((packed));

static struct idt_entry idt[256];
extern uint32_t isr_stub_table[32];     /* addresses of isr0..isr31, from isr.s */

void idt_set_gate(uint8_t n, uint32_t handler, uint16_t sel, uint8_t flags)
{
    idt[n].offset_low  = handler & 0xFFFF;
    idt[n].offset_high = handler >> 16;
    idt[n].selector    = sel;
    idt[n].zero        = 0;
    idt[n].type_attr   = flags;
}

void idt_init(void)
{
    for (int i = 0; i < 32; i++)
        idt_set_gate(i, isr_stub_table[i], 0x08, 0x8E);
    struct idt_ptr ip = { sizeof(idt) - 1, (uint32_t)&idt };
    __asm__ volatile ("lidt %0" : : "m"(ip));
}
```

Task 3.4 Write the interrupt stubs

This is the most important assembly in the project; every line carries a comment. Two NASM macros generate the 32 stubs: one for vectors where the CPU pushes an error code and one that pushes a dummy zero, so every frame has the same shape. All stubs jump to one common routine that builds the rest of Figure 3.

kernel/arch/i386/isr.s

```
extern isr_handler

%macro ISR_NOERR 1
isr%1:
    push dword 0            ; dummy error code: keeps every frame the same shape
    push dword %1           ; the vector number
    jmp  isr_common
%endmacro

%macro ISR_ERR 1
isr%1:
    push dword %1           ; the CPU has already pushed an error code
    jmp  isr_common
%endmacro

ISR_NOERR 0
ISR_NOERR 1
ISR_NOERR 2
ISR_NOERR 3
ISR_NOERR 4
ISR_NOERR 5
ISR_NOERR 6
ISR_NOERR 7
ISR_ERR   8
ISR_NOERR 9
ISR_ERR   10
ISR_ERR   11
ISR_ERR   12
ISR_ERR   13
ISR_ERR   14
ISR_NOERR 15
ISR_NOERR 16
ISR_ERR   17
ISR_NOERR 18
ISR_NOERR 19
ISR_NOERR 20
ISR_ERR   21
ISR_NOERR 22                ; 22..31: reserved, no error code
ISR_NOERR 23
ISR_NOERR 24
ISR_NOERR 25
ISR_NOERR 26
ISR_NOERR 27
ISR_NOERR 28
ISR_NOERR 29
ISR_NOERR 30
ISR_NOERR 31

isr_common:
    pusha                   ; saves eax, ecx, edx, ebx, esp, ebp, esi, edi
    mov  ax, ds
    push eax                ; save the interrupted code's data segment
    mov  ax, 0x10           ; switch to the kernel data segment
    mov  ds, ax
    mov  es, ax
    mov  fs, ax
    mov  gs, ax
    push esp                ; argument for isr_handler: pointer to the frame
    call isr_handler
    add  esp, 4             ; drop the argument
    pop  eax                ; restore the original data segment
    mov  ds, ax
```

```
    mov  es, ax
    mov  fs, ax
    mov  gs, ax
    popa
    add  esp, 8             ; drop int_no and err_code
    iret                    ; pop eip, cs, eflags and resume

global isr_stub_table       ; table of stub addresses for idt_init()
isr_stub_table:
%assign i 0
%rep 32
    dd isr%+i
%assign i i+1
%endrep
```

Task 3.5 The C exception handler

Mirror the frame in a struct, lowest address first. Then write `isr_handler`: if a handler is registered for the vector (next task), call it and return; otherwise print a crash report and panic. For a page fault, also print CR2, which holds the faulting address.

include/arch/regs.h and kernel/arch/i386/isr.c

```
struct regs {
    uint32_t ds;                                    /* pushed last by isr_common */
    uint32_t edi, esi, ebp, esp_at_pusha, ebx, edx, ecx, eax;   /* pusha */
    uint32_t int_no, err_code;                      /* pushed by the stub        */
    uint32_t eip, cs, eflags;                       /* pushed by the CPU         */
    uint32_t useresp, ss;                           /* only from ring 3 (Phase 10) */
};

static const char *exception_names[32] = {
    "Divide error", "Debug", "NMI", "Breakpoint", "Overflow", "Bound range exceeded",
    "Invalid opcode", "Device not available", "Double fault", "Coprocessor overrun",
    "Invalid TSS", "Segment not present", "Stack-segment fault", "General protection",
    "Page fault", "Reserved", "x87 FPU error", "Alignment check", "Machine check",
    "SIMD FP exception", "Virtualization", "Control protection", /* 22..31 */ };

void isr_handler(struct regs *r)
{
    if (handlers[r->int_no]) {              /* registered handler: let it deal with it */
        handlers[r->int_no](r);
        return;
    }
    const char *name = exception_names[r->int_no] ? exception_names[r->int_no] : "Reserved";
    kprintf("\n*** CPU exception %lu: %s (error code 0x%lx)\n", r->int_no, name, r->err_code);
    kprintf("EIP=%08lx  CS=%04lx  EFLAGS=%08lx\n", r->eip, r->cs, r->eflags);
    kprintf("EAX=%08lx  EBX=%08lx  ECX=%08lx  EDX=%08lx\n", r->eax, r->ebx, r->ecx, r->edx);
    kprintf("ESI=%08lx  EDI=%08lx  EBP=%08lx\n", r->esi, r->edi, r->ebp);
    if (r->int_no == 14) {
        uint32_t cr2;
        __asm__ volatile ("mov %%cr2, %0" : "=r"(cr2));
        kprintf("CR2 (faulting address) = %08lx\n", cr2);
    }
    panic("unhandled CPU exception");
}
```

Verify: `sizeof(struct regs)` is 64 bytes (16 fields). Assert it at compile time with `_Static_assert`.

Portability note

Every field of `struct regs` is an x86 register. In Phase 16 it becomes `struct trapframe`, one per architecture, and code outside `kernel/arch/i386` reads it only through small helpers: the system call number, an argument, the return value, the program counter. The convention applies from this phase on: leave field names like `eax`, `eip` and `cs` to the x86 code, and when a later phase (page faults, system calls, signals) needs a register elsewhere, give it a one-line inline helper instead of a field name.

Task 3.6 A handler registration API

Add `register_interrupt_handler(uint8_t n, void (*fn)(struct regs *))` backed by a 256-entry table of function pointers. Phase 4 installs the timer and keyboard handlers through it, Phase 6 the page-fault handler, and Phase 10 the system call handler. Register a breakpoint handler now that prints "breakpoint at EIP=..." and simply returns.

Task 3.7 Trigger every reachable exception

Add a `crash` command to the boot sequence (a real command monitor arrives in Phase 4). Write one tiny function per exception and check that each gives the expected report. The `volatile` stops the compiler from noticing the division by zero at compile time.

`tests/exceptions.c`

```
void test_divide_error(void)  { volatile int zero = 0; volatile int x = 1 / zero; (void)x; }
void test_invalid_opcode(void) { __asm__ volatile ("ud2"); }
void test_breakpoint(void)     { __asm__ volatile ("int3"); kprintf("...and we continued!\n"); }
void test_gp(void)             { __asm__ volatile ("mov $0x2B, %ax; mov %ax, %ds"); } /* entry 5 is still
    ↪ empty */
```

Verify: Divide error, invalid opcode and general protection each print a report with a sensible EIP (check it with `i686-elf-addr2line -e build/kernel.elf <eip>`). The breakpoint test prints both messages and the kernel keeps running.

5 Testing and debugging

Two QEMU tools are essential here. `-d int -no-reboot` logs every interrupt and exception with its vector (`v=0d` is #GP, `v=0e` is #PF) and the error code, and stops instead of rebooting on a triple fault. The monitor's `info registers` shows the GDT and IDT base and limit. With Bochs installed, its debugger can also decode the tables with `info gdt` and `info idt`.

Symptom	Likely cause	Resolution
Reboot right after `lgdt`	Wrong limit (should be `sizeof(gdt)-1`), struct not packed, or the far jump is missing	Check `sizeof(struct gdt_entry) == 8`; look at `GDT=` in `info registers`.
#GP immediately after loading DS or SS	Selector points at the wrong entry, or the access byte is wrong	`0x10` must be index 2 with access `0x92`; print the table bytes.
The divide-error report prints forever	Returning from a fault re-executes the faulting instruction	Faults must be fixed or must end in `panic`; only traps return safely.
Crash or garbage after `iret`	Unbalanced stack: missing `add esp, 8`, or a stub pushed the wrong number of values	Every path must pop exactly what it pushed; step through `isr_common` in GDB.
`int_no` or registers look shuffled	`struct regs` field order doesn't match the push order	The struct lists fields from the *last* push to the first.

Symptom	Likely cause	Resolution
Double fault (vector 8) on every exception	The handler itself faults, for example because of a stack overflow	Use `-d int` to see the first exception in the chain; that one is the real bug.

6 Exit criteria

Exit criteria: No more silent reboots

- ☐ `info registers` shows the kernel GDT (limit `0x2f`) and IDT (limit `0x7ff`), with CS=`0x08` and DS=`0x10`.
- ☐ Divide by zero, `ud2` and a bad segment load each produce a named crash report with registers, and no reboot.
- ☐ `int3` prints a message and execution continues after it.
- ☐ `register_interrupt_handler` works, and `sizeof(struct regs)` is checked at compile time.
- ☐ Every line of `isr_common` is commented with its effect on the stack.

Review questions

1. If the flat model makes segmentation do nothing, why is a GDT still required, and why does it need ring 3 entries?
2. Decode selector `0x23`: which GDT entry, which table, which RPL?
3. What does an interrupt gate do to the IF flag that a trap gate does not? Why does Kagan want that?
4. Which exceptions push an error code, and why do the other stubs push a dummy value?
5. What does the stack contain, from top to bottom, at the moment `isr_handler` is called?
6. What distinguishes a fault, a trap and an abort? Give one example of each.
7. Why does returning from the divide-error handler run the handler again?

7 Extensions

Print a backtrace in the crash report by walking the chain of saved EBP values: `[ebp]` holds the caller's EBP and `[ebp+4]` the return address. Because the kernel is compiled with `-fno-omit-frame-pointer`, the chain is intact. Print raw addresses for now and symbolise them on the host with `addr2line`; Phase 15 can embed a symbol table. A deeper extension gives the double fault its own known-good stack through a task gate, so even a kernel stack overflow produces a report. That trick needs the TSS machinery from Phase 10.

Next: **Phase 4 Hardware Interrupts: PIC, Timer and Keyboard** →

References

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at https://littleosbook.github.io.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at https://pages.cs.wisc.edu/~remzi/OSTEP/.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

LOB and OSTEP are freely available; the roadmap lists ISBNs for the others. Where sources disagree, the design follows the newer one when its method also suits Kagan. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free OSTEP chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in OSDI Appendix B.